NEXT ENGINEER · _SHEERAT_

Blockchain Architecture Design

BCS063

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, CS, IT | **Semester:** 6 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 3rd year, Departmental Elective, 2024-25 onwards

UNIT 1 Introduction to Blockchain and its Architecture

UNIT 2 Consensus and Permissioned Blockchains

UNIT 3 Hyperledger Fabric

UNIT 4 Blockchain in Financial Services and Supply Chain

UNIT 5 Blockchain in Government and Blockchain Security

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: architecture, consensus protocols and comparison tables.
3. **Solved examples**: hashing and avalanche effect, Merkle trees, proof-of-work difficulty, Byzantine fault tolerance counts, Fabric transaction flow and chaincode, and use-case designs.
4. **Key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (digital money to distributed ledgers, design primitives, hash, signatures, hash chain, Merkle tree, block structure) and Unit 2 (consensus requirements, proof of work, scalability, permissioned blockchains, PBFT and Raft).
- **Afternoon (3 hours):** Unit 3 (Fabric architecture, execute-order-validate, components, chaincode, SDK, Composer).
- **Evening (2 hours):** Unit 4 (settlements, KYC, capital markets, insurance, supply chain, trade finance, invoice discounting) and Unit 5 (digital identity, land records, PDS, cryptography, privacy and security). Then the "Last-hour checklist".
- **In the exam:** draw the block structure, the hash chain, the Merkle tree and the Fabric transaction flow. For use-case questions, give: problem today → blockchain design (participants, data on chain, smart contracts) → benefits → challenges.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to Blockchain and its Architecture

✍️ 2-mark definitions

- **Blockchain:** a distributed, append-only ledger of transactions grouped into blocks, where each block contains the hash of the previous block, so that past records cannot be changed without detection; copies are kept by many nodes that agree on the contents through consensus.
- **Distributed ledger technology (DLT):** a database replicated and synchronized across many nodes without a central administrator. Blockchain is one type of DLT.
- **Digital money:** money that exists in electronic form. Early schemes (DigiCash, e-gold) needed a trusted central party.
- **Double spending:** spending the same digital token twice; the main problem that digital cash must solve. Bitcoin solved it without a central party through a public ledger and proof of work.
- **Cryptographic hash function:** a function that maps input of any size to a fixed-size output (SHA-256 gives 256 bits) and is deterministic, fast, pre-image resistant (one-way), second pre-image resistant and collision resistant, with an avalanche effect.
- **Digital signature:** a value created with a private key that proves who signed a message and that it was not changed; anyone can verify it with the public key (ECDSA in Bitcoin and Ethereum).
- **Hash pointer:** a pointer to where data is stored together with the hash of that data, so that changes can be detected.
- **Hash chain:** a sequence of records where each contains the hash of the previous one; changing any record breaks all later links.
- **Merkle tree:** a binary tree of hashes where leaves are transaction hashes and each parent is the hash of its two children; the root summarizes all transactions.
- **Genesis block:** the first block of a blockchain, with no previous block.
- **Node:** a computer participating in the blockchain network: full node (stores the whole chain and validates), light node (stores headers only), miner/validator (creates blocks).
- **Permissionless (public) blockchain:** anyone can join, read, transact and take part in consensus (Bitcoin, Ethereum).
- **Permissioned (private or consortium) blockchain:** only identified, authorized participants can join and validate (Hyperledger Fabric, R3 Corda, Quorum).
- **Smart contract:** a program stored on the blockchain that runs automatically when its conditions are met; in Fabric it is called chaincode.

💡 Core concepts

From digital money to distributed ledgers

1. Centralized e-cash (David Chaum's DigiCash, 1990s): privacy with blind signatures, but needed a bank.
2. Proposals for decentralized money: b-money (Wei Dai), Bit Gold (Nick Szabo), Hashcash proof of work (Adam Back, 1997, against spam).

3. **Bitcoin (Satoshi Nakamoto, white paper 2008, network launched January 2009):** combined a public ledger, proof of work, digital signatures and economic incentives to make peer-to-peer cash without a trusted third party.
4. **Ethereum (proposed 2013 by Vitalik Buterin, launched 2015):** a programmable blockchain with Turing-complete smart contracts.
5. **Enterprise DLT (2015 onwards):** Hyperledger (Linux Foundation, 2015), Corda, Quorum: permissioned ledgers for businesses, focusing on identity, privacy and performance.

Generations: Blockchain 1.0 (cryptocurrency), 2.0 (smart contracts, DeFi), 3.0 (enterprise and government applications, scalability, interoperability).

Design primitives

- **Protocols:** rules for how nodes communicate, propagate transactions and blocks (peer-to-peer gossip), validate them, and maintain the chain.
- **Security:** cryptographic hashing (integrity), digital signatures (authentication and non-repudiation), and consensus (resistance to malicious nodes); immutability through hash chaining.
- **Consensus:** the mechanism by which distributed nodes agree on the same order and content of transactions (proof of work, proof of stake, PBFT, Raft).
- **Permissions:** who can read, write and validate (public vs private, permissionless vs permissioned); membership services and certificates in permissioned systems.
- **Privacy:** pseudonymous addresses in public chains (not true anonymity); in enterprise chains: private channels, private data collections, zero-knowledge proofs, encryption.

Blockchain architecture (layers): hardware/infrastructure → data layer (blocks, transactions, Merkle trees, hash chain) → network layer (P2P, gossip) → consensus layer → incentive layer (in public chains: block rewards, fees) → contract layer (smart contracts, scripts) → application layer (wallets, DApps, enterprise apps).

Block structure

Part	Field	Purpose
Header	Version	Rules followed
Header	Previous block hash	Links to the previous block (the chain)
Header	Merkle root	Summary of all transactions in the block
Header	Timestamp	Time of block creation
Header	Difficulty target (bits)	Proof-of-work target
Header	Nonce	Number miners change to find a valid hash
Body	Transaction counter and list of transactions	The actual data

The **block hash** is the hash of the header. Since the header includes the previous hash and the Merkle root, changing one transaction changes the Merkle root, then the block hash, then the next block's previous-hash field, and so on: the tamper becomes visible.

Crypto primitives in detail

- **Hash properties:** deterministic; fixed length; fast to compute; pre-image resistance (given h, finding x with H(x) = h is infeasible); second pre-image resistance; collision resistance; avalanche effect (a tiny

change in input completely changes the output); puzzle-friendliness (used in proof of work).

- **Public-key cryptography:** a key pair (private key kept secret, public key shared); addresses are derived from public keys by hashing.
- **Digital signature process:** sender computes H(m) and signs it with the private key: sig = Sign(sk, H(m)); receiver checks Verify(pk, H(m), sig). Properties: authenticity, integrity, non-repudiation.

Hash chain to blockchain: a hash chain links records; a blockchain extends this by grouping transactions into blocks, using Merkle trees inside blocks, using consensus to decide which block is next, and replicating the chain on many nodes. The longest valid chain (most cumulative work) is accepted in proof-of-work systems.

Merkle tree benefits: a single root commits to all transactions; **Merkle proof** (proof of inclusion) needs only log2(n) hashes, which allows light clients (SPV, simplified payment verification) to check a transaction without downloading the whole block.

Basic consensus mechanisms (overview): proof of work (miners solve a hash puzzle), proof of stake (validators chosen according to their stake), delegated proof of stake (token holders vote for delegates), proof of authority (approved validators), practical Byzantine fault tolerance (voting among known nodes), Raft (leader-based crash fault tolerance).

Public vs private vs consortium blockchain

Feature	Public	Private	Consortium
Who can join	Anyone	One organization decides	A group of organizations
Identity	Pseudonymous	Known	Known
Consensus	PoW, PoS	Raft, PBFT, PoA	PBFT, Raft
Speed	Slow (Bitcoin about 7 transactions per second)	Fast (thousands per second)	Fast
Transparency	Fully public	Restricted	Restricted to members
Examples	Bitcoin, Ethereum	Internal bank ledger	Hyperledger Fabric networks, Corda, IBM Food Trust

🧮 Solved examples

Example 1 (avalanche effect with SHA-256).

- SHA-256("Next Engineer") = 2f809c3130baf428...aea4bd07
- SHA-256("Next engineer") = cd3d23c58d1260b1...5e8f2fed

Changing one letter's case changes the whole hash. This is why altering a record in a block is detected immediately.

Example 2 (Merkle tree with 4 transactions). H1 = H(T1), H2 = H(T2), H3 = H(T3), H4 = H(T4); H12 = H(H1 ‖ H2), H34 = H(H3 ‖ H4); root = H(H12 ‖ H34).

- To prove T3 is in the block, a light client needs only **H4 and H12** (2 = log2 4 hashes): it computes H3, then H34, then the root, and compares with the root in the block header.
- For a block with 4,096 transactions, a proof needs only log2 4096 = **12 hashes**.

Example 3 (proof of work). Find a nonce such that SHA-256("prev=0000a1|tx=A→B:5|nonce=N") starts with 4 hexadecimal zeros.

- Each hex digit has 16 possibilities, so the expected number of attempts is 16^4 = **65,536**.
- Trying N = 0, 1, 2, ... the first valid nonce is **N = 26,369**, giving hash 0000d5728607d750... 68ac766a. Anyone can verify it with one hash computation (hard to find, easy to check).
- Adding one more required zero multiplies the expected work by 16.

Example 4 (tampering cost). An attacker changes a transaction in block 100 when the chain is at block 106. They must redo proof of work for block 100 and all 6 following blocks faster than the honest network adds new blocks. With less than 50% of the hash power, the chance of catching up falls exponentially with the number of blocks (this is why exchanges wait for about 6 confirmations in Bitcoin).

Key points to remember

- Bitcoin white paper 2008; network 2009; Ethereum 2015; Hyperledger 2015.
- Hash properties: pre-image, second pre-image, collision resistance, avalanche, puzzle-friendly.
- Block header: version, previous hash, Merkle root, timestamp, difficulty, nonce.
- Merkle proof size = log2(n) hashes.
- PoW expected attempts for d leading hex zeros = 16^d.
- Public vs private vs consortium table.

⭐ Important questions

1. Trace the journey from digital money to distributed ledgers. What is double spending and how does blockchain prevent it?
2. Explain the design primitives of blockchain: protocols, security, consensus, permissions and privacy.
3. Explain the blockchain architecture and the structure of a block.
4. Explain cryptographic hash functions and digital signatures and their role in blockchain.
5. Explain how a hash chain becomes a blockchain. Explain Merkle trees and Merkle proofs.
6. Compare public, private and consortium blockchains.

UNIT 2 Consensus and Permissioned Blockchains

✍️ 2-mark definitions

- **Consensus:** the process by which nodes in a distributed system agree on a single value or a single order of transactions, even when some nodes fail or behave maliciously.
- **Byzantine fault:** a fault in which a node behaves arbitrarily or maliciously (sending conflicting messages), not just stopping.
- **Crash fault:** a node simply stops responding.
- **Byzantine Generals Problem (Lamport, Shostak, Pease, 1982):** generals must agree on a plan even though some may be traitors; agreement is possible only if fewer than one-third are traitors ($n \geq 3f + 1$).
- **FLP impossibility:** in a fully asynchronous network, no deterministic protocol can guarantee consensus if even one node can crash; practical systems use timeouts, randomization or partial synchrony.
- **Proof of Work (PoW):** consensus where nodes compete to solve a computational puzzle; the winner adds the next block and earns a reward.
- **Mining:** the process of collecting transactions, building a block and searching for a nonce that satisfies the PoW target.
- **51% attack:** an attacker controlling more than half of the hash power can reverse recent transactions (double spend) and censor others.
- **Fork:** a split of the chain into two branches; temporary forks resolve by the longest-chain rule; soft forks (backward-compatible rule changes) and hard forks (incompatible changes) are protocol upgrades.
- **Proof of Stake (PoS):** consensus where validators are chosen to create blocks in proportion to the cryptocurrency they lock (stake); misbehaviour is punished by losing stake (slashing).
- **Finality:** the guarantee that a confirmed transaction will not be reversed: probabilistic in PoW, immediate (deterministic) in BFT protocols.
- **Scalability trilemma:** a blockchain finds it hard to be decentralized, secure and scalable at the same time.
- **Permissioned blockchain:** a blockchain where participants are known and authorized, so faster consensus protocols such as PBFT or Raft can be used.
- **PBFT (Practical Byzantine Fault Tolerance, Castro and Liskov, 1999):** a voting protocol that tolerates f Byzantine nodes among $3f + 1$, with three phases: pre-prepare, prepare and commit.
- **Raft:** a leader-based crash-fault-tolerant consensus protocol (leader election, log replication); tolerates f crashed nodes among $2f + 1$.

💡 Core concepts

Requirements for consensus protocols

1. **Agreement (safety/consistency):** all honest nodes decide the same value.

2. **Validity:** the decided value was proposed by some node and is valid.
3. **Termination (liveness):** all honest nodes eventually decide.
4. **Fault tolerance:** works despite crash or Byzantine faults.
5. **Integrity:** a node decides at most once.

Also desirable: performance (throughput, latency), scalability (number of nodes), energy efficiency, and fairness.

Proof of Work (Bitcoin)

- Miners collect transactions, build a block header, and repeatedly change the nonce until SHA-256(SHA-256(header)) < target.
- **Difficulty adjustment:** every 2,016 blocks (about two weeks) the target is adjusted so that the average block time stays near 10 minutes.
- **Reward:** block subsidy (halves every 210,000 blocks, about four years; 3.125 BTC since the April 2024 halving) plus transaction fees.
- **Longest chain rule:** nodes accept the valid chain with the most cumulative work.
- **Advantages:** open participation, proven security for public networks, Sybil resistance (identities cost computing power).
- **Disadvantages:** huge energy use, slow (about 7 transactions per second, 10-minute blocks), probabilistic finality, mining centralization in pools, 51% attack risk.

Scalability approaches: bigger or faster blocks (trade-off with decentralization); off-chain/Layer 2 (Lightning Network payment channels, rollups); sharding (split the network into parallel chains); alternative consensus (PoS, BFT); sidechains; DAG-based ledgers.

Proof of Stake and others: PoS (Ethereum since "The Merge", September 2022) uses far less energy; validators deposit stake (32 ETH per validator in Ethereum) and are randomly selected; dishonest validators lose stake. Delegated PoS (EOS), Proof of Authority (known validators, private networks), Proof of Elapsed Time (Intel SGX, Hyperledger Sawtooth), Proof of Burn, Proof of Capacity.

Permissioned blockchains: design goals

1. **Known identities:** participants are identified by certificates (accountability, KYC compliance).
2. **Access control:** fine-grained permissions for reading, writing, validating and deploying contracts.
3. **Performance:** high throughput and low latency using efficient consensus without mining.
4. **Privacy and confidentiality:** data shared only with parties who need it (channels, private data).
5. **Finality:** immediate and deterministic, needed in finance.
6. **Governance:** clear rules for joining, upgrading and resolving disputes.
7. **Modularity and pluggability:** choose the consensus, identity and database components.
8. **Compliance and auditability:** regulators can audit; data retention rules respected.

Consensus protocols for permissioned blockchains

Protocol	Fault type	Nodes needed for f faults	Idea	Used in
Paxos	Crash	2f + 1	Proposers, acceptors, learners; majority quorums	Google Chubby, distributed databases

Protocol	Fault type	Nodes needed for f faults	Idea	Used in
Raft	Crash	2f + 1	Elected leader replicates a log to followers; majority commit	Hyperledger Fabric ordering service, etcd
PBFT	Byzantine	3f + 1	Pre-prepare, prepare, commit phases with 2f + 1 matching votes	Early Fabric (v0.6), many BFT systems
Tendermint / IBFT / QBFT	Byzantine	3f + 1	Rounds of propose, prevote, precommit with rotating proposers	Cosmos, Quorum, Besu
Kafka-based ordering (older Fabric)	Crash	Kafka cluster with ZooKeeper	Ordered log from Kafka	Fabric v1.x (deprecated)

PBFT phases

1. **Request:** client sends a request to the primary (leader).
2. **Pre-prepare:** the primary assigns a sequence number and multicasts it to all replicas.
3. **Prepare:** each replica multicasts a prepare message; when a replica has 2f matching prepares plus the pre-prepare, it is "prepared".
4. **Commit:** replicas multicast commit messages; with 2f + 1 matching commits, the request is committed and executed.
5. **Reply:** replicas reply to the client; the client waits for f + 1 matching replies. - **View change:** if the primary is faulty or slow, replicas elect a new primary. - Message complexity $O(n^2)$, so PBFT suits tens of nodes, not thousands.

Raft: nodes are leader, follower or candidate. A follower that hears no heartbeat within a random election timeout becomes a candidate and requests votes; a majority makes it leader for that term. The leader appends client entries to its log and replicates them; an entry is committed once stored on a majority, then applied.

🧮 Solved examples

Example 1 (BFT node count). To tolerate f = 2 Byzantine nodes, PBFT needs n ≥ 3 × 2 + 1 = **7 nodes**; a quorum is 2f + 1 = **5** matching messages. To tolerate 2 crash faults, Raft needs 2 × 2 + 1 = **5 nodes**, with a majority of 3.

Example 2 (how many faults can a network survive?). A consortium has 10 validating nodes running PBFT. f = floor((10 − 1)/3) = **3 Byzantine nodes**. With Raft and 10 nodes (better to use an odd number), f = floor((10 − 1)/2) = 4 crash faults.

Example 3 (Bitcoin numbers). 10-minute blocks, about 2,000 to 3,000 transactions per block: throughput ≈ 2500/600 ≈ **4 to 7 transactions per second**, compared with thousands per second for card networks. For 6 confirmations a user waits about **1 hour**.

Example 4 (block reward after halvings). Initial reward 50 BTC (2009); halvings in 2012 (25), 2016 (12.5), 2020 (6.25), 2024 (**3.125 BTC**). The total supply is capped at 21 million BTC.

Key points to remember

- Consensus requirements: agreement, validity, termination, fault tolerance.
- Byzantine: $n \geq 3f + 1$; crash: $n \geq 2f + 1$.
- PoW: 10-minute blocks, difficulty every 2,016 blocks, halving every 210,000 blocks; 51% attack; probabilistic finality.
- Permissioned design goals: identity, access control, performance, privacy, finality, governance.
- PBFT: request, pre-prepare, prepare, commit, reply; view change; $O(n^2)$.
- Raft: leader election, log replication, majority commit.

⭐ Important questions

1. What are the requirements of a consensus protocol? Explain the Byzantine Generals Problem.
2. Explain proof of work in detail. Discuss its advantages, disadvantages and the 51% attack.
3. Explain the scalability problem of blockchain and the approaches to solve it.
4. Explain the design goals of permissioned blockchains.
5. Explain consensus protocols for permissioned blockchains: PBFT and Raft. Compare them with PoW.

NEXT ENGI
sheerat

UNIT 3 Hyperledger Fabric

✍️ 2-mark definitions

- **Hyperledger:** an open-source umbrella project of the Linux Foundation (started 2015) for enterprise blockchain frameworks and tools (Fabric, Besu, Indy, Iroha, Sawtooth, Caliper).
- **Hyperledger Fabric:** a modular, permissioned blockchain framework (originally contributed by IBM and Digital Asset) for enterprise use, with pluggable consensus and identity, and chaincode in general-purpose languages.
- **Peer:** a node that hosts ledgers and chaincode. **Endorsing peers** execute chaincode and sign results; **committing peers** validate blocks and update the ledger; **anchor peers** let organizations discover each other.
- **Orderer (ordering service):** nodes that order endorsed transactions into blocks and deliver them to peers (Raft in current versions).
- **Membership Service Provider (MSP):** the component that defines which identities (X.509 certificates issued by a Certificate Authority) belong to an organization and what roles they have.
- **Fabric CA:** the default Certificate Authority that issues identities to members.
- **Channel:** a private sub-network with its own ledger, shared only by the members of that channel.
- **Ledger:** in Fabric, the **blockchain** (immutable log of blocks) plus the **world state** (current values of keys, stored in LevelDB or CouchDB).
- **Chaincode:** Fabric's smart contract, written in Go, Java or JavaScript/TypeScript, which reads and writes the world state.
- **Endorsement policy:** a rule stating which organizations must endorse (sign) a transaction for it to be valid, for example "AND('Org1.peer', 'Org2.peer')".
- **Read-write set (RW set):** the keys and versions a transaction read and the keys and values it wants to write, produced during simulation.
- **Private data collection:** data shared only among a subset of organizations on a channel; other members see only its hash.
- **Fabric SDK:** client libraries (Node.js, Java, Go) used by applications to submit transactions and query the ledger.
- **Hyperledger Composer:** an earlier (now deprecated) tool set for modelling business networks and generating chaincode and REST APIs quickly.

💡 Core concepts

Decomposing consensus: execute-order-validate Traditional blockchains use **order-execute** (order transactions first, then every node executes them sequentially), which limits throughput and needs deterministic code. Fabric splits consensus into three phases:

1. **Execute (endorsement):** the client sends a transaction proposal to endorsing peers chosen by the endorsement policy. Each peer simulates the chaincode against its world state, without updating it, and returns a signed proposal response with the read-write set.

2. **Order:** the client collects enough endorsements, packages them into a transaction and sends it to the ordering service, which orders transactions from all clients into blocks (it does not look at the contents).
3. **Validate (commit):** each committing peer checks every transaction in the block: endorsement policy satisfied (VSCC) and no read conflicts (MVCC: the versions read must still be current). Valid transactions update the world state; invalid ones are marked invalid but remain in the block. Peers then notify clients by events.

Benefits: parallel execution, non-deterministic code is caught (endorsements must match), confidentiality (only endorsers run the chaincode), and a pluggable ordering service.

Fabric components (architecture)

- **Organizations** with their **MSPs** and **CAs**.
- **Peers** (endorsing, committing, anchor) holding ledgers and chaincode.
- **Ordering service** (Raft cluster of orderers).
- **Channels** with their ledgers and configuration blocks.
- **Client applications** using the SDK or Gateway.
- **Chaincode** installed on peers and defined on channels (lifecycle: package → install → approve by each organization → commit to the channel).
- **World state database:** LevelDB (key-value, default) or CouchDB (JSON documents, rich queries).
- **Gossip protocol:** peers share blocks and private data efficiently.

Fabric vs Ethereum

Feature	Hyperledger Fabric	Ethereum (public)
Type	Permissioned	Permissionless
Identity	Known (X.509 certificates via MSP)	Pseudonymous addresses
Consensus	Execute-order-validate with Raft ordering	Proof of stake
Smart contracts	Chaincode in Go, Java, JavaScript	Solidity/Vyper on the EVM
Cryptocurrency	None required	Ether needed for gas fees
Privacy	Channels, private data	All transactions public
Throughput	Thousands of transactions per second	Tens per second on the base layer

Chaincode design and implementation

- Chaincode implements business logic as functions (transactions) that read and write key-value pairs: `GetState`, `PutState`, `DelState`, `GetStateByRange`, `GetQueryResult` (CouchDB rich query), `GetHistoryForKey`, and `GetPrivateData` / `PutPrivateData`.
- Design guidelines: keep chaincode deterministic (no random numbers, current time or external API calls inside); use composite keys for structured lookups; validate inputs and the caller's identity (client identity library, attribute-based access control); emit events for applications; keep transactions small; version and upgrade through the lifecycle with endorsement policies.

Beyond chaincode: Fabric SDK and front end

- The application uses the SDK or the Fabric Gateway API: load a wallet identity → connect to a gateway (peer) → get the network (channel) → get the contract → `submitTransaction` (endorse, order, commit) or `evaluateTransaction` (query only, no ordering).
- Front end: a web or mobile app calls a REST API (Node.js/Express or Java Spring) that uses the SDK; events update the user interface.

Hyperledger Composer (deprecated in 2019, still in syllabus)

- Modelling language (.cto files) to define **assets, participants, transactions and events**; transaction logic in JavaScript; access control rules (.acl); queries (.qry). These were packaged into a business network archive (.bna).
- Tools: Composer Playground (browser-based testing), REST server (generates REST APIs automatically), Yeoman generator for Angular front ends.
- Benefit: fast prototyping by business developers. It was replaced by direct Fabric development (contract APIs) and later tools.

🧮 Solved examples

Example 1 (Fabric transaction flow for a car transfer). Org1 (dealer) and Org2 (registration office) share a channel; endorsement policy AND(Org1, Org2).

1. The buyer's app (Org1 client) sends proposal `TransferCar(CAR10, "Ravi")` to peer1.org1 and peer1.org2.
2. Both simulate: read CAR10 (version 4), write CAR10.owner = "Ravi"; both return the same RW set with signatures.
3. The app sends the endorsed transaction to the orderer; Raft orders it into block 52.
4. All peers validate: both endorsements present (policy met) and CAR10 still at version 4 (no conflict) → valid; world state updated to owner = Ravi (version 5).
5. If another transaction in the same block also changed CAR10 earlier, this one fails the MVCC check and is marked invalid.

Example 2 (chaincode in Go).

NEXT ENGINEER _sheerat_

```
package main

import (
    "encoding/json"
    "fmt"
    "github.com/hyperledger/fabric-contract-api-go/contractapi"
)

type Land struct {
    ID    string `json:"id"`
    Owner string `json:"owner"`
    Area  int    `json:"area"`
}

type LandContract struct{ contractapi.Contract }

func (c *LandContract) Register(ctx contractapi.TransactionContextInterface, id, owner string,
area int) error {
    existing, err := ctx.GetStub().GetState(id)
    if err != nil {
        return err
    }
    if existing != nil {
        return fmt.Errorf("land %s already registered", id)
    }
    data, _ := json.Marshal(Land{ID: id, Owner: owner, Area: area})
    return ctx.GetStub().PutState(id, data)
}

func (c *LandContract) Transfer(ctx contractapi.TransactionContextInterface, id, newOwner
string) error {
    data, err := ctx.GetStub().GetState(id)
    if err != nil || data == nil {
        return fmt.Errorf("land %s not found", id)
    }
    var l Land
    if err := json.Unmarshal(data, &l); err != nil {
        return err
    }
    l.Owner = newOwner
    out, _ := json.Marshal(l)
    return ctx.GetStub().PutState(id, out)
}

func (c *LandContract) Query(ctx contractapi.TransactionContextInterface, id string) (*Land,
error) {
    data, err := ctx.GetStub().GetState(id)
    if err != nil || data == nil {
        return nil, fmt.Errorf("land %s not found", id)
    }
    var l Land
    err = json.Unmarshal(data, &l)
    return &l, err
}

func main() {
    cc, err := contractapi.NewChaincode(&LandContract{})
```

```
    if err != nil {
        panic(err)
    }
    if err := cc.Start(); err != nil {
        panic(err)
    }
}
```

Example 3 (client application with the Node.js SDK, outline).

```
const gateway = new Gateway();
await gateway.connect(connectionProfile, { wallet, identity: 'appUser', discovery: { enabled:
true } });
const network  = await gateway.getNetwork('landchannel');
const contract = network.getContract('landcc');
await contract.submitTransaction('Transfer', 'PLOT-17', 'Neha');      // endorse, order,
commit
const result   = await contract.evaluateTransaction('Query', 'PLOT-17'); // query only
console.log(JSON.parse(result.toString()));
gateway.disconnect();
```

Example 4 (Composer model, .cto).

```
namespace org.land
participant Citizen identified by aadhaarHash { o String name }
asset Plot identified by plotId { o String plotId  o Double area  --> Citizen owner }
transaction TransferPlot { --> Plot plot  --> Citizen newOwner }
```

Key points to remember

- Execute (endorse, simulate, RW set) → order (Raft) → validate (endorsement policy + MVCC) → commit.
- Peers: endorsing, committing, anchor; orderers; MSP and CA; channels; private data.
- Ledger = blockchain + world state (LevelDB or CouchDB).
- Chaincode lifecycle: package, install, approve, commit. Keep chaincode deterministic.
- SDK: submitTransaction vs evaluateTransaction.
- Composer: .cto model, JavaScript logic, .acl, .bna, REST server (deprecated).

⭐ Important questions

1. Explain the architecture of Hyperledger Fabric and its components.
2. Explain how Fabric decomposes consensus into execute-order-validate. Draw the transaction flow.
3. Explain chaincode design and implementation with an example.
4. Explain the Fabric SDK and how a front end interacts with the network.
5. Write a note on Hyperledger Composer.
6. Compare Hyperledger Fabric and Ethereum.

UNIT 4 Blockchain in Financial Services and Supply Chain

✍️ 2-mark definitions

- **Financial software and systems (FSS):** the systems used by banks and financial institutions for payments, settlements, trading, lending, KYC and insurance.
- **Settlement:** the final transfer of money or securities between parties to complete a trade or payment.
- **KYC (Know Your Customer):** the process by which financial institutions verify the identity and address of customers to prevent fraud and money laundering.
- **Capital markets:** markets for buying and selling long-term securities (shares, bonds).
- **Tokenization:** representing a real-world asset (shares, bonds, property, invoices) as a digital token on a blockchain.
- **Provenance:** the origin and history of ownership and movement of a product.
- **Supply chain visibility:** the ability of all participants to see the status and location of goods as they move.
- **Trade finance:** financing of international trade transactions, such as letters of credit, bills of lading and guarantees.
- **Letter of credit (LC):** a bank's guarantee that the seller will be paid once the agreed documents are presented.
- **Invoice discounting (factoring):** a business sells or borrows against its unpaid invoices to get cash before the buyer pays.
- **Double financing:** fraud where the same invoice is used to borrow from several financiers.

💡 Core concepts

Settlements

- **Today:** trades pass through brokers, clearing houses and custodians; settlement takes T+1 or T+2 days (India moved to T+1 for equities in 2023); cross-border payments go through correspondent banks and SWIFT messages and can take days with high fees; each party reconciles its own records.
- **With blockchain:** a shared ledger records the trade once for all parties; smart contracts perform delivery versus payment (DvP: securities and cash move together, atomically); near real-time settlement; fewer intermediaries; less reconciliation; tokenized cash or central bank digital currency (India's e-rupee pilot) for the money leg.
- **Examples:** JPMorgan's Onyx (Kinexys) for interbank payments, Project Ubin (Singapore), cross-border pilots by central banks (mBridge).

KYC

- **Problems today:** each bank repeats KYC for the same customer (duplicate effort and cost), customers submit documents again and again, data is in silos, and updates do not propagate.
- **Blockchain KYC:** a consortium of banks shares a KYC ledger; the first bank verifies the customer and stores a hash or encrypted reference to the documents; the customer gives consent for other banks

to access it; updates are visible to all with an audit trail. Benefits: lower cost, faster onboarding, consent-based privacy, regulator access. In India, the Central KYC Registry (CKYCR) already centralizes KYC records; blockchain proposals aim to decentralize and secure such sharing.

- **Challenges:** data privacy laws (store only hashes on chain; personal data off chain), liability for errors, interoperability, governance.

Capital markets

- Applications: tokenized securities and bonds (faster issuance, fractional ownership), post-trade clearing and settlement, corporate actions (dividends, voting automated by smart contracts), syndicated loans, collateral management, transparent registries of share ownership.
- Benefits: shorter settlement, lower counterparty risk, transparency for regulators, 24×7 markets.
- Example: tokenized bond issues on DLT by banks and governments; the Australian Securities Exchange's planned CHESS replacement on DLT was cancelled in 2022, showing implementation risk.

Insurance

- **Problems:** fraud (duplicate claims across insurers), slow claim processing, heavy paperwork, reinsurance reconciliation.
- **Blockchain uses:** shared claims registry to detect duplicate claims; **parametric insurance** where a smart contract pays automatically when an oracle reports an event (for example flight delay above 2 hours, or rainfall below a threshold for crop insurance); reinsurance contracts with shared data (the B3i initiative); IoT data for motor and health insurance.
- India example: discussions of a blockchain-based shared repository among insurers (Insurance Information Bureau pilots).

Supply chain management

- **Problems today:** many participants (farmers, manufacturers, logistics, distributors, retailers) with separate records; poor visibility; counterfeits; slow recalls; paperwork in shipping.
- **Provenance:** each step (harvest, processing, packing, shipping, retail) is recorded with time, location, certificates and handlers, often captured by IoT sensors and QR or RFID tags; consumers scan a code to see the history. Example: IBM Food Trust (Walmart traced the source of mangoes in 2.2 seconds instead of about 7 days).
- **Visibility:** all authorized parties see shipment status in real time; disputes and delays reduce. Example: TradeLens (Maersk and IBM, shut down in 2023 because the wider industry did not join, an important lesson about consortium adoption).
- **Benefits:** traceability, anti-counterfeiting (pharmaceuticals, luxury goods), faster recalls, compliance (organic, fair trade), automated payments on delivery.
- **Limitations:** "garbage in, garbage out" (the blockchain cannot verify that the physical product matches the record; needs trusted IoT and audits), cost of onboarding small suppliers, data-sharing reluctance.

Trade finance

- **Problems today:** paper documents (letters of credit, bills of lading, invoices) sent by courier; manual checking; weeks of delay; fraud through forged or duplicate documents.
- **Blockchain solution:** buyer, seller, their banks, shipper, customs and insurer share a ledger. A smart contract represents the letter of credit; documents (digital bill of lading) are uploaded and verified; when shipment is confirmed (by the shipper or IoT), payment is released automatically.

- **Examples:** Contour, we.trade and Marco Polo (pilots, several since closed); in India, a consortium of banks used a blockchain platform (IBBIC, Indian Banks' Blockchain Infrastructure Company) for inland letters of credit.

Invoice management and discounting

- **Problems:** invoices verified manually; double financing; delayed payments hurt MSMEs.
- **Blockchain solution:** the supplier registers an invoice (its hash) on a shared ledger; the buyer confirms it; financiers see whether it is already financed, preventing double financing; a smart contract tracks financing, repayment and settlement. Tokenized invoices can be sold in parts to several investors.
- **India link:** TReDS (Trade Receivables Discounting System, regulated by RBI) platforms already provide invoice discounting for MSMEs; blockchain can add a shared, tamper-proof registry across platforms.

🧮 Solved examples

Example 1 (design: blockchain KYC consortium).

- Participants: banks, customers, the regulator (RBI as an observer), KYC verification agencies.
- On chain: customer ID hash, verification status, verifying bank, timestamp, consent records. Off chain: encrypted documents in each bank's secure store.
- Smart contracts: `RegisterKYC`, `GrantConsent(bank)`, `RevokeConsent`, `UpdateAddress`, `FlagSuspicious`.
- Benefits: customer verified once, used by many; audit trail; consent control. Challenges: privacy law, who pays, liability.

Example 2 (parametric flight-delay insurance). Premium ₹200; payout ₹5,000 if the flight is delayed more than 2 hours. A smart contract subscribes to an oracle for flight status. If the oracle reports a delay of 2 h 40 min, the contract pays ₹5,000 to the traveller's wallet automatically; no claim form is needed.

Example 3 (double financing check). Supplier S registers invoice INV-881 (₹10 lakh) with hash h. Financier F1 funds it and marks h as "financed". When S approaches F2 with the same invoice, F2 checks h on the ledger, sees it is already financed, and refuses. The fraud is prevented without F1 and F2 sharing their full customer data.

Example 4 (supply-chain provenance for a medicine). Manufacturer records batch B123 (ingredients, test certificate) → distributor records receipt and storage temperature from an IoT logger → pharmacy records receipt → patient scans the QR code and sees the full history; a counterfeit pack would have no valid history.

Key points to remember

- Settlement: DvP, T+1 today vs near real time on DLT; tokenized cash or CBDC.
- KYC: verify once, share with consent; store hashes on chain, documents off chain.
- Insurance: shared claims registry, parametric contracts with oracles.
- Supply chain: provenance, visibility, recall; IBM Food Trust; TradeLens lesson.
- Trade finance: digital LC and bill of lading; IBBIC in India.

- Invoice discounting: prevent double financing; TReDS.

⭐ Important questions

1. How can blockchain improve settlements in financial services?
2. Explain the use of blockchain in KYC with a design.
3. Discuss blockchain applications in capital markets and insurance.
4. Explain how blockchain provides provenance and visibility in supply chains, with examples and limitations.
5. Explain blockchain in trade finance, invoice management and invoice discounting.

NEXT ENGINEER
sheerat

UNIT 5 Blockchain in Government and Blockchain Security

✍️ 2-mark definitions

- **Digital identity:** an electronic representation of a person or organization used to prove who they are in online transactions.
- **Self-sovereign identity (SSI):** a model where individuals hold and control their own identity credentials in a digital wallet and share only what is needed, using decentralized identifiers (DIDs) and verifiable credentials.
- **Verifiable credential:** a tamper-proof digital certificate (degree, licence, proof of age) issued and signed by an authority and verifiable by anyone.
- **Land records:** official records of who owns a piece of land, its boundaries, and the history of transfers and encumbrances (mortgages).
- **Public Distribution System (PDS):** India's system for distributing subsidized food grains to eligible households through fair price shops.
- **Zero-knowledge proof (ZKP):** a method by which one party proves a statement is true without revealing any other information (for example proving age > 18 without revealing the birth date).
- **Ring signature:** a signature that proves a message was signed by some member of a group without revealing which one (used in Monero).
- **Homomorphic encryption:** encryption that allows computations on encrypted data.
- **Sybil attack:** creating many fake identities to gain influence in a network.
- **Eclipse attack:** isolating a node by controlling all its peer connections so that it sees a false view of the blockchain.
- **Smart contract vulnerability:** a bug in contract code that attackers can exploit, such as re-entrancy (the 2016 DAO attack on Ethereum, about 3.6 million ETH drained).
- **Oracle problem:** blockchains cannot verify external data by themselves; the oracle that supplies it becomes a point of trust.

💡 Core concepts

Digital identity

- **Problems today:** identity data held in many central databases (risk of breaches), repeated verification, identity theft, people without documents excluded.
- **Blockchain approach:** issuers (government, university) issue verifiable credentials to a citizen's wallet; the issuer's public keys and DIDs are anchored on a blockchain; verifiers check signatures and revocation status on the ledger; the citizen shares only required attributes (selective disclosure, ZKPs).
- **Benefits:** user control, privacy, fewer central honeypots, instant verification. Examples: Estonia's e-identity (uses KSI blockchain technology for data integrity), EU digital identity wallet, India's DigiLocker (centralized today, but a similar idea of verified documents).

Land records

- **Problems:** paper records, forgery, multiple sales of the same plot, disputes clogging courts, slow mutation (transfer of records), corruption.
- **Blockchain solution:** each plot's title and every transfer recorded on a permissioned ledger shared by the revenue department, registration office, banks and courts; smart contracts check ownership and encumbrances before registering a sale; history cannot be altered; citizens can verify ownership online.
- **Examples:** Georgia (with Bitfury, 2016), Sweden's Lantmäteriet pilot, Andhra Pradesh and Telangana pilots in India, and NITI Aayog studies.
- **Limitations:** the existing records must first be digitized and corrected ("garbage in"); legal recognition of blockchain records; integration with courts.

Inter-government agency record keeping

- Different departments (tax, police, health, municipal, education) keep separate databases; data does not flow; citizens submit the same documents repeatedly.
- A permissioned blockchain gives a shared, consistent record with access rights by role, an audit trail of every access and change, and smart contracts for workflows (for example automatic verification of birth certificates by the education department).
- Benefits: transparency, less duplication, faster services, reduced corruption and fraud.

Public distribution system and social welfare

- **Problems:** leakage (grain diverted to the open market), fake or duplicate ration cards, ghost beneficiaries, manipulation of stock records.
- **Blockchain design:** track grain from Food Corporation of India warehouses → state depots → fair price shops → beneficiary, with each handover recorded and verified (weighbridges, IoT, biometric authentication of beneficiaries); smart contracts reconcile stock and release subsidies; citizens and auditors can see allocations.
- **Social welfare (DBT, pensions, scholarships):** eligibility rules in smart contracts; payments traced end to end; duplicate beneficiaries detected; audit by social auditors.
- **Challenges:** connectivity in rural areas, biometric failures excluding genuine beneficiaries, integration with Aadhaar and existing systems, privacy of beneficiaries.

Blockchain cryptography (recap and extras): SHA-256 and Keccak-256 hashes; ECDSA and EdDSA signatures; elliptic curve secp256k1 (Bitcoin, Ethereum); Merkle trees; multi-signature (m-of-n approval); threshold signatures; ZKPs (zk-SNARKs in Zcash and rollups); commitment schemes; post-quantum cryptography as a future need.

Privacy in blockchain

- **Issues:** public ledgers show all transactions; addresses are pseudonymous but can be linked to real identities by analysis; enterprise data must not be visible to competitors; regulations like GDPR and India's DPDP Act include a right to erasure, which conflicts with immutability.
- **Techniques:** permissioned access; Fabric channels and private data collections (only hashes on the shared ledger); off-chain storage with on-chain hashes; encryption; ZKPs; mixing, ring signatures and stealth addresses (privacy coins); confidential transactions.

Security of blockchain

Layer	Threats	Defences
Consensus	51% attack, selfish mining, long-range attacks (PoS), nothing-at-stake	High decentralization, more confirmations, checkpoints, slashing
Network	Sybil, eclipse, DDoS, routing (BGP) attacks	Diverse peers, PoW/PoS costs, permissioned membership, rate limits
Smart contract	Re-entrancy, integer overflow, access-control bugs, front-running	Audits, formal verification, safe libraries, testing, bug bounties
Keys and wallets	Private key theft, phishing, lost keys	Hardware wallets, multi-signature, secure key management (HSM)
Oracle	False external data	Multiple independent oracles, reputation, signed data
Application / exchange	Exchange hacks, insider threats	Cold storage, audits, regulation

🧮 Solved examples

Example 1 (design: land registry on Fabric).

- Organizations: revenue department, sub-registrar office, banks, courts (observer).
- On chain: plot ID, owner ID hash, area, survey number, encumbrances, transaction history. Off chain: scanned deeds and maps, with hashes on chain.
- Chaincode: `RegisterPlot`, `Transfer` (requires seller signature, buyer signature and sub-registrar endorsement; checks no encumbrance), `AddMortgage`, `ReleaseMortgage`, `Query`.
- Endorsement policy: AND(Revenue, Registrar); banks endorse mortgage transactions.
- Benefits: no double sale, instant verification for banks, transparent history. See the Go chaincode in Unit 3, Example 2.

Example 2 (PDS tracking). 10,000 kg of rice leaves a depot for 20 fair price shops. Each shop records receipt (weight from a connected scale). Shop 7 records 450 kg instead of 500 kg; the smart contract flags the 50 kg shortfall between dispatch and receipt for investigation. Beneficiaries authenticate by fingerprint; each distribution is recorded, and monthly totals are reconciled automatically.

Example 3 (ZKP intuition). A citizen must prove they are over 18 to buy a service. With a verifiable credential and a ZKP, the wallet proves "birth date < 26 Oct 2008" to the verifier without revealing the actual date or other details on the ID.

Key points to remember

- Digital identity: SSI, DIDs, verifiable credentials, selective disclosure.
- Land records: title history, double-sale prevention, digitization first.
- Inter-agency records: shared ledger, role-based access, audit trail.
- PDS: end-to-end tracking, reconcile stock, detect leakage; challenges of exclusion.
- Cryptography: SHA-256, ECDSA (secp256k1), Merkle, multisig, ZKPs.

- Privacy: channels, private data, off-chain storage, ZKPs; conflict with right to erasure.
- Security table: consensus, network, contract, keys, oracle, application.

⭐ Important questions

1. Explain the use of blockchain for digital identity (self-sovereign identity).
2. Explain blockchain-based land records management with a design.
3. How can blockchain help inter-government agency record keeping?
4. Explain the use of blockchain in the public distribution system and social welfare schemes.
5. Explain blockchain cryptography and the techniques used for privacy.
6. Discuss security threats to blockchain systems and their defences.

Last-hour checklist

- ☐ Digital money history; double spending; Bitcoin 2008/2009; Ethereum; Hyperledger.
- ☐ Design primitives: protocols, security, consensus, permissions, privacy.
- ☐ Hash properties and avalanche example; digital signature steps.
- ☐ Block header fields; hash chain; Merkle tree and proof (log2 n); PoW example (16^4 = 65,536; nonce 26,369).
- ☐ Public vs private vs consortium.
- ☐ Consensus requirements; Byzantine generals; FLP; $n \geq 3f + 1$ vs $2f + 1$.
- ☐ PoW details, difficulty, halving (3.125 BTC), 51% attack, scalability approaches, PoS.
- ☐ Permissioned design goals; PBFT phases; Raft.
- ☐ Fabric components; execute-order-validate; MVCC; endorsement policy; chaincode lifecycle; SDK submit vs evaluate; Composer.
- ☐ Settlements, KYC, capital markets, insurance, supply chain, trade finance, invoice discounting (with examples).
- ☐ Digital identity, land records, inter-agency records, PDS and welfare.
- ☐ Cryptography, privacy techniques, security threats table.

Books prescribed by AKTU

These are the books and resources named in the official AKTU syllabus. Buy or borrow the books through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Mastering Bitcoin: Unlocking Digital Cryptocurrencies	Andreas M. Antonopoulos	O'Reilly	WorldCat
2	Blockchain: Blueprint for a New Economy	Melanie Swan	O'Reilly	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
3	Hyperledger Fabric documentation	Hyperledger Foundation	Linux Foundation (online)	hyperledger-fabric.readthedocs.io
4	Zero to Blockchain: An IBM Redbooks course	Bob Dill, David Smits	IBM Redbooks (online course)	IBM Redbooks

"Mastering Bitcoin" is published by its author under an open licence and can be read free on GitHub (github.com/bitcoinbook). The Bitcoin white paper (bitcoin.org/bitcoin.pdf) is a free, short and very useful read for Units 1 and 2.

Syllabus source: AKTU B.Tech 3rd year CS/IT syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER _sheerat_